Zindi Study Jam Series 002: Unido Africa Rice Quality Assessment Challenge

Uriel Nguefack Yefou

- Bachelor in Applied Physics

- Master of engineering in Industrial Robotic from NHPSD

- Master of science in Data Science from AIMS Cameroon

- Master in Artificial Intelligence from the AMMI program at AIMS Senegal

- PhD student in AI and Cybersecurity, Concordia University, Montreal, Canada

- Zindi Country Ambassador, Cameroon

- LinkedIn: uriel-nguefack-Yefou

- Organizations: Yulcom Technologies, OIF, MHI, Kamer Engineering Solutions (KES), ENEO, LABRED

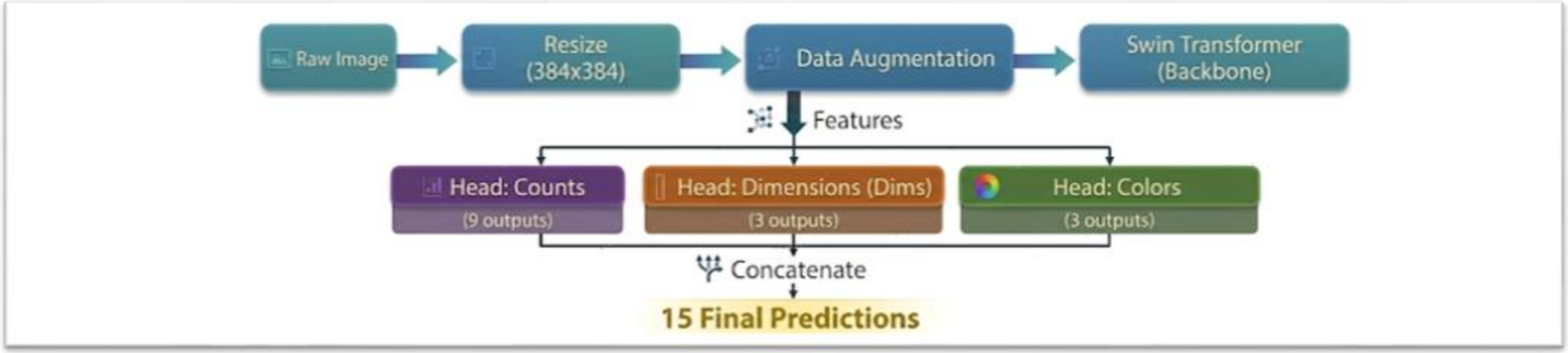
Understanding the Problem

- **Task:** Multi-output regression from images of rice samples
- **Input:** Image of a rice tray
- **Output:** 15 targets simultaneously:
  - Count-based (grain counts by category: Broken, Long, Medium, Black, Chalky, Red, Yellow, Green, Total)
  - Dimensional averages (WK_Length, WK_Width, WK_LW_Ratio)
  - Color averages (L, a, b channels)
- **Metric:** Mean Absolute Error (MAE), averaged across all 15 targets
- Show a sample image if you have one

Why Is This Hard?

- **Scale mismatch**: `Count` ≈ 2000, `WK_LW_Ratio` ≈ 0.5 - naive loss is dominated by large values
- **15 targets at once:** a single model needs to handle very different output types
- **Counting is vision-hard:** requires fine-grained spatial understanding
- **Domain rules:** e.g., Paddy rice has no chalky grains; counts must be integers ≥ 0

My Pipeline at a Glance (Architecture Overview)




```python
def seed_everything(seed=42):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.backends.cudnn.deterministic = True

seed_everything(CONFIG['seed'])
```

Reproducibility

Model Choice: Why Swin Transformer?

- Started with `EfficientNet-B3` (baseline), then upgraded to `swin_base_patch4_window12_384`
- Swin was pre-trained at **384×384** (native resolution match = less information loss)
- Transformers capture **global context** well, important for counting grains across the whole image
- Used **ImageNet-22K pre-trained weights** via timm transfer learning matters a lot here
- Final attempt also tested `EVA-02` (a stronger ViT variant)

Data Augmentation

```python
def get_transforms(data='train'):
    if data == 'train':
        return A.Compose([
            A.Resize(CONFIG['img_size'], CONFIG['img_size']),
            A.HorizontalFlip(p=0.5),
            A.VerticalFlip(p=0.5),
            A.RandomRotate90(p=0.5),
            # Cutout helps the model focus on counting, not just memorizing texture
            A.CoarseDropout(max_holes=8, max_height=32, max_width=32, p=0.3),
            A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
            ToTensorV2(),
        ])
    elif data == 'valid':
        return A.Compose([
            A.Resize(CONFIG['img_size'], CONFIG['img_size']),
            A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
            ToTensorV2(),
        ])
```

- **Why CoarseDropout?** Forces the model to count from partial views, not memorize patterns
- Augmentations are symmetric (rotation invariant) because rice trays have no "up" orientation

Loss Function & Target Scaling (Critical Lessons)

Target Scaling:

•Use StandardScaler on all 15 targets before training
•Without this, Count ≈ 2000 dominates the loss and the model ignores LW_Ratio ≈ 0.5
•Inverse-transform after inference to get real values back

Loss Function:

•Started with L1Loss (MAE) good, matches the metric
•Switched to HuberLoss(delta=1.0): more robust to outlier sample
•Huber = L1 for large errors, L2 for small errors → best of both worlds

Training Strategy

Setting	Value	Why
Image size	384×384	Native Swin resolution
Batch size	32	Stable gradient estimates
Optimizer	AdamW	Weight decay regularization
Scheduler	CosineAnnealingLR	Smooth LR decay
Epochs	100	With early patience
Cross-validation	5-Fold KFold	Robust OOF score
Mixed precision	autocast + GradScaler	2× faster, less VRAM

Test Time Augmentation

```python
def tta_inference(dataloader, model, device):
    model.eval()
    all_preds = []

    with torch.no_grad():
        for images in tqdm(dataloader, desc="TTA Inference"):
            images = images.to(device)

            # 1. Original
            pred_1 = model(images)

            # 2. Horizontal Flip
            pred_2 = model(torch.flip(images, [-1]))

            # 3. Vertical Flip
            pred_3 = model(torch.flip(images, [-2]))

            # 4. Rotate 90 degrees clockwise
            pred_4 = model(torch.rot90(images, k=1, dims=[-2, -1]))

            # 5. Rotate 180 degrees
            pred_5 = model(torch.rot90(images, k=2, dims=[-2, -1]))

            # 6. Rotate 270 degrees (90 counter-clockwise)
            pred_6 = model(torch.rot90(images, k=3, dims=[-2, -1]))

            # 7. Horizontal Flip + 90 rotation
            pred_7 = model(torch.rot90(torch.flip(images, [-1]), k=1, dims=[-2, -1]))

            # 8. Vertical Flip + 90 rotation
            pred_8 = model(torch.rot90(torch.flip(images, [-2]), k=1, dims=[-2, -1]))

            # Average all predictions
            avg_pred = (pred_1 + pred_2 + pred_3 + pred_4 + pred_5 + pred_6 + pred_7 + pred_8) / 8.0
            all_preds.append(avg_pred.cpu().numpy())

            # Memory cleanup for large batches
            del pred_1, pred_2, pred_3, pred_4, pred_5, pred_6, pred_7, pred_8, avg_pred

    return np.concatenate(all_preds)
```

- At inference, instead of predicting once, predict 8 **times** per image
- **Average the 8 predictions** → smoother, more robust output
- Typically gives **1–2% MAE improvement** with no retraining

Post-Processing & Domain Rules

This is where **domain knowledge** beats raw ML:

1. Round all Count columns → integers (you can't have 5.7 grains)

2. Clip all Count columns → ≥ 0 (negative counts are impossible)

3. For Paddy samples: - Chalky_Count = 0 (Paddy rice has no chalky grains by definition) - Count = Black + Red + Yellow + Green (total is sum of color counts)

What I Would Do Differently

- **Ensemble multiple models** (EfficientNet + Swin + EVA-02 + ConvNext): averaging predictions reduces variance
- **Higher resolution:** 1024 or 2048 for count-heavy tasks, memory permitting
- **Pseudo-labeling:** use your best model to label the test set and retrain
- **Custom loss weights:** weight the count outputs higher since they're the hardest targets
- **Explore stratified splits:** split by Comment (Paddy vs Milled) to ensure balanced folds
- **Separated models:** Train 3 models to predict the head count, head dimensions, and head colors, and merge the results after inference.
- Use multiple GPUs.

Resources & Tools Used

- timm — model zoo (Swin, EfficientNet, EVA-02)
- albumentations — fast augmentation library
- PyTorch + torch.cuda.amp — mixed precision training
- scikit-learn — StandardScaler, KFold
- Jupyter/Kaggle/Colab for GPU access

Key Takeaways for Beginners

- **Start simple:** a resnet baseline tells you a lot
- **EDA:** Display some images to see what information you can get.
- **Understand your metric:** MAE here means all targets are equally important
- **Scale your targets:** don't let one variable dominate
- **Use pre-trained models:** don't train from scratch on small datasets
- **Post-process with domain knowledge:** rules about Paddy are free points
- **TTA is free performance:** always try it at inference
- Train with multiple "epochs" and add "patience" and "learning rate scheduler".
- **Hyperparameter tuning:** Do an hyperparameter tuning on lr, batch size, optimizer,...

Open Computer Vision challenges on Zindi

ZiND! Google databricks

The AI4EAC Agriculture Practice Challenge

- **Host: Makerere AI Lab, Marconi Machine Learning Lab and Makerere University Research Innovations Fund (RIF) & Zindi**
- **Description: Can you develop a mobile-friendly machine learning model to identify diseases on cocoa leaves?**
- **Prizes**
- There are 1 500 Zindi points available per track.
- **Why interesting: Computer Vision, Agriculture, Object detection**
- **Status:** Open until March 27, 2026
- **Link to competition:** https://zindi.africa/competitions/the-ai4eac-agriculture-practice-challenge

Bean Flowering Detection Challenge

- **Host: Makerere AI Lab, Marconi Machine Learning Lab and Makerere University Research Innovations Fund (RIF) & Zindi**
- **Description: Can you build a computer vision system to detect bean plants and classify flowering intensity?**
- **Prizes**
- There are $ 7 500 USD to share for the 3 first winners.
- **Why interesting: Computer Vision, Agriculture, Object detection**
- **Status:** Open until March 29, 2026
- **Link to competition:** https://zindi.africa/competitions/bean-flowering-detection-challenge

ZiNDI Google databricks

Are you interested in being a changemaker, sign up?

https://zindi.africa/signup

@zindi.africa

@ZindiAfrica

@Zindi